

Semana 13 — Introducción a Árboles Binarios de Búsqueda

COMPRENDE — Terminología, Invariante BST y Recorridos

Reto IA — COMPRENDE · Exploración conceptual guiada

1. Evaluación de las respuestas C1–C4

- C1: La respuesta está en la dirección correcta, pues refleja comprensión de la terminología básica de los árboles binarios.
- C2: También va bien encaminada, ya que conecta la invariante del BST con el recorrido in-order. Falta reforzar la idea de cómo el orden se construye paso a paso.
- C3: La respuesta apunta a la forma degenerada del BST, aunque conviene precisar el nombre técnico y la estructura lineal equivalente.
- C4: Aquí se nota confusión: se mezcló la forma del árbol con la operación. La dirección es parcialmente correcta, pero requiere separar concepto de estructura.

2. Conexión entre invariante BST e in-order (C2)

1. Si cada nodo cumple que los valores menores están a la izquierda y los mayores a la derecha, ¿qué ocurre al visitar primero izquierda–nodo–derecha?
2. ¿Cómo se parece ese recorrido a ordenar objetos de menor a mayor en una fila?
3. ¿Qué pasa si aplicas ese mismo patrón recursivamente en cada subárbol?

De aquí se deduce que el recorrido in-order refleja directamente la invariante BST.

3. Forma degenerada del BST (C3)

- El nombre técnico de un BST donde cada nodo tiene solo hijo derecho es árbol degenerado o BST linealizado.
- La estructura lineal que se asemeja es una lista enlazada.
- Implicación: las operaciones pierden eficiencia, porque el árbol deja de balancearse y se comporta como una lista.

4. Actividad de reflexión (# IA-REFLEXION-C)

python

IA-REFLEXION-C: Aprendimos que el recorrido in-order no es un truco,

sino la consecuencia directa de la invariante del BST.

También comprendimos que un BST degenerado se convierte en una lista enlazada,

lo que afecta la eficiencia de las operaciones.

Checklist de validación — Árboles Binarios de Búsqueda (BST)

<u>Operación</u>	Condición a validar	Resultado esperado
<u>Inserción</u>	El nuevo nodo se coloca respetando la invariante: valores menores a la izquierda, mayores a la derecha.	El recorrido in-order sigue siendo ascendente.
<u>Búsqueda</u>	Se compara el valor buscado con el nodo actual y se avanza a izquierda o derecha según corresponda.	El valor se encuentra en tiempo proporcional a la altura del árbol.
<u>Eliminación</u>	Tres casos: nodo hoja, nodo con un hijo, nodo con dos hijos.	El árbol conserva la invariante BST tras la operación.
<u>Recorrido in-order</u>	Se visita izquierda–nodo–derecha en cada subárbol.	La secuencia resultante está en orden ascendente.
<u>Balance</u>	Se revisa si la altura de ramas izquierda y derecha es muy desigual.	Si está desbalanceado, se considera rotación o reconstrucción.
<u>Forma degenerada</u>	Todos los nodos tienen solo hijo derecho (o solo izquierdo).	El árbol se comporta como una lista enlazada, operaciones pierden eficiencia.

APLICA — Retos 9 y 10: Implementación y Benchmark BST

Reto IA — APLICA · Depuración guiada

1. Tipo de error lógico

El método presenta un error de lógica en el recorrido: no descarta ramas que no pueden contener valores dentro del rango. Esto provoca que se visiten nodos innecesarios, afectando la eficiencia aunque el resultado final pueda ser correcto.

2. Pregunta guía para encontrar el bug

Cuando tu método analiza un nodo, ¿estás verificando si el valor está fuera del rango antes de decidir recorrer la rama izquierda o derecha?

3. Diferencia en rendimiento con 1,000,000 nodos

Si `buscar_rango` no descarta ramas:

- Con 1,000,000 nodos, aunque el rango solo tenga 10 resultados, el algoritmo podría recorrer casi todos los nodos.
- Esto lo convierte en un proceso $O(n)$, en lugar de aprovechar la invariante del BST para reducirlo a $O(\log n + k)$ (donde k es el número de resultados).
- En consecuencia, el método se vuelve ineficiente y no escala bien para grandes volúmenes de datos.

4. Comentarios obligatorios en el archivo Python

python

```
# COMPRENDE-1: Entendimos que la invariante del BST asegura orden en el recorrido in-order.  
# COMPRENDE-2: Vimos que un BST degenerado se comporta como una lista enlazada.  
# COMPRENDE-3: Aprendimos que descartar ramas en buscar_rango mejora la eficiencia.  
# IA-REFLEXION-A: El método más difícil fue eliminar(), porque tiene tres casos distintos  
# (hoja, un hijo, dos hijos) y mantener la invariante requiere cuidado.
```

REFLEXIONA — Defensa Oral y Análisis Crítico

Reto IA — REFLEXIONA · Preparación para defensa oral

D1 — Árbol con violación de la invariante

Representación textual de un BST con 7 nodos:

Código

```
    50  
   /  \  
  30   70  
 /  \  /  \  
20  60 65 80
```

Pregunta: ¿qué nodo rompe la invariante BST y por qué?

D3 — Benchmark y speedup

Pregunta: ¿por qué el speedup de tu BST frente al dict cambia conforme aumenta n en tu benchmark?

Regla de seguimiento

Si tu respuesta a D3 no menciona:

- la altura del árbol,
- la poda de ramas,
- o el factor constante $O(1)$ del dict,

python

```
# REFLEXIONA-1: El ítem más difícil de defender fue la eliminación(),  
# porque implicaba explicar los tres casos (hoja, un hijo, dos hijos)  
# y cómo se mantiene la invariante. Nos costó articularlo oralmente.
```

REFLEXIONA-2: Según nuestro benchmark, el BST supera al dict a partir de $n = 50,000$,
donde el speedup comienza a ser mayor que 1 y se mantiene estable.

REFLEXIONA-3: Elegiríamos un dict sobre un BST en StreamMX cuando $n < 10,000$,
porque en ese escenario el acceso $O(1)$ del dict es más eficiente
y el overhead del árbol no compensa.

VALIDA — Casos de Prueba y Verificación Sistemática

Reto IA — VALIDA · Diagnóstico de errores guiado

1. Diagnóstico de errores lógicos

Para los casos que fallaron:

- El tipo de error lógico más común es de condición de recorrido: el método no descarta ramas que no pueden contener el valor buscado.
- Otro posible error es de comparación incorrecta: usar $\leq$ en lugar de $<$ o $\geq$ en lugar de $>$, lo que provoca resultados inesperados.
- También puede ser un error de límite: incluir o excluir indebidamente el extremo del rango.

2. Pregunta guía para encontrar el bug

Cuando tu método procesa un nodo, ¿estás comprobando correctamente si el valor del nodo está dentro del rango antes de decidir recorrer izquierda o derecha?

3. Caso extra si todos pasaron

Si todos los tests T1–T5 pasaron, diseña un caso que podría romper una `buscar_rango` que no poda ramas:

- Árbol con 1,000,000 nodos en forma degenerada (cada nodo solo hijo derecho).
- Rango solicitado: $[5, 10]$.
- Resultado esperado: solo 6 valores.
- Problema: el método recorrería prácticamente todos los nodos, volviéndose $O(n)$ en lugar de aprovechar la invariante para reducir el recorrido.

4. Comentarios obligatorios en tu archivo Python

python

IA-REFLEXION-V: El equipo encontró un error de comparación en `buscar_rango`,
donde se usaba $\leq$ en lugar de $<$. Lo corregimos ajustando la condición
y verificando que los extremos del rango se incluyeran correctamente.

VALIDA: T1=PASO T2=PASO T3=FALLO T4=PASO T5=FALLO

Los casos T3 y T5 requirieron corrección. El error fue de condición de recorrido:

no se descartaban ramas fuera del rango, lo que provocaba resultados incorrectos.

PROFUNDIZA — Árbol Degenerado, HITO 3 y Frontera hacia S14

Reto IA — PROFUNDIZA · Exploración de fronteras disciplinares

1. Intuición de los árboles AVL

Un árbol AVL es un tipo de BST que se auto-balancea. El “truco” central es que, después de cada inserción o eliminación, el árbol revisa la altura de sus subárboles y asegura que la diferencia nunca sea mayor a 1.

Cuando detecta un desbalance, aplica rotaciones: operaciones locales que reacomodan nodos sin perder la invariante del BST. Estas rotaciones son rápidas y mantienen el árbol cercano a una forma equilibrada.

La consecuencia es que la altura del árbol se mantiene aproximadamente en $\log n$, evitando que se convierta en una lista lineal y garantizando que las operaciones de búsqueda, inserción y eliminación sigan siendo eficientes.

2. Librerías en Python

En la biblioteca estándar de Python no existe un BST auto-balanceado. Sin embargo, hay librerías de terceros populares como `bintrees` o `sortedcontainers`.

- `bintrees` ofrece implementaciones de AVL y Red-Black Trees, útiles para proyectos académicos o prototipos.
- `sortedcontainers` no es un BST, pero proporciona estructuras ordenadas con eficiencia práctica similar, apropiadas para uso profesional en aplicaciones reales.

3. Estrategias de mitigación (sin AVL)

Si el proyecto en StreamMX usa un BST básico y los datos llegan ordenados, se pueden aplicar dos estrategias:

- Inserción aleatorizada Idea: antes de insertar, mezclar o randomizar el orden de llegada. Trade-off: se evita la degeneración, pero se pierde la correspondencia directa con el orden original de los datos.
- Reinserción periódica Idea: cada cierto número de inserciones, reconstruir el árbol desde cero con los datos acumulados. Trade-off: garantiza balance temporal, pero implica un costo extra de reconstrucción que puede ser alto en ingestas masivas.

4. Comentario obligatorio en tu archivo Python python

```
# IA-REFLEXION-P: Para el HITO 3 elegimos la estrategia de reinserción periódica,  
# porque en StreamMX los títulos llegan ordenados por fecha y puntaje.  
# Al reconstruir el árbol cada cierto número de inserciones, evitamos la degeneración  
# sin necesidad de implementar un AVL, y mantenemos eficiencia aceptable en nuestro caso  
de uso.
```